

DBMS PROJECT · BML MUNJAL UNIVERSITY · 2026

OBSYRIA

Video Game Market
Intelligence System

Database Management Systems

Guided by Dr. Nishtha Phutela

TEAM

Reia Baid 240602
Abhishek Mishra 240365
Parth Malik 240571
Advit Sharma 240372

100

Steam Games Tracked

9

SQL Features Implemented

5

Platforms Compared

3

Custom DB Functions

The Problem

01

No Centralized Analytics

The gaming industry has no unified platform for cross-storefront market intelligence.

02

ROI Blind Spot

Publishers lack clear visibility into return-on-investment across competing storefronts.

03

No Scale Classification

No standardized method to classify games as Indie, AA, or AAA at scale.

04

Slow Manual Research

Market analysis remains manual, fragmented, and impossible to update in real time.

OBSYRIA unifies these four gaps into a single, live, SQL-powered intelligence platform.

What We Built

01

Real Steam

Populated from
real games, n

FINANCIAL YIELD (ROI)

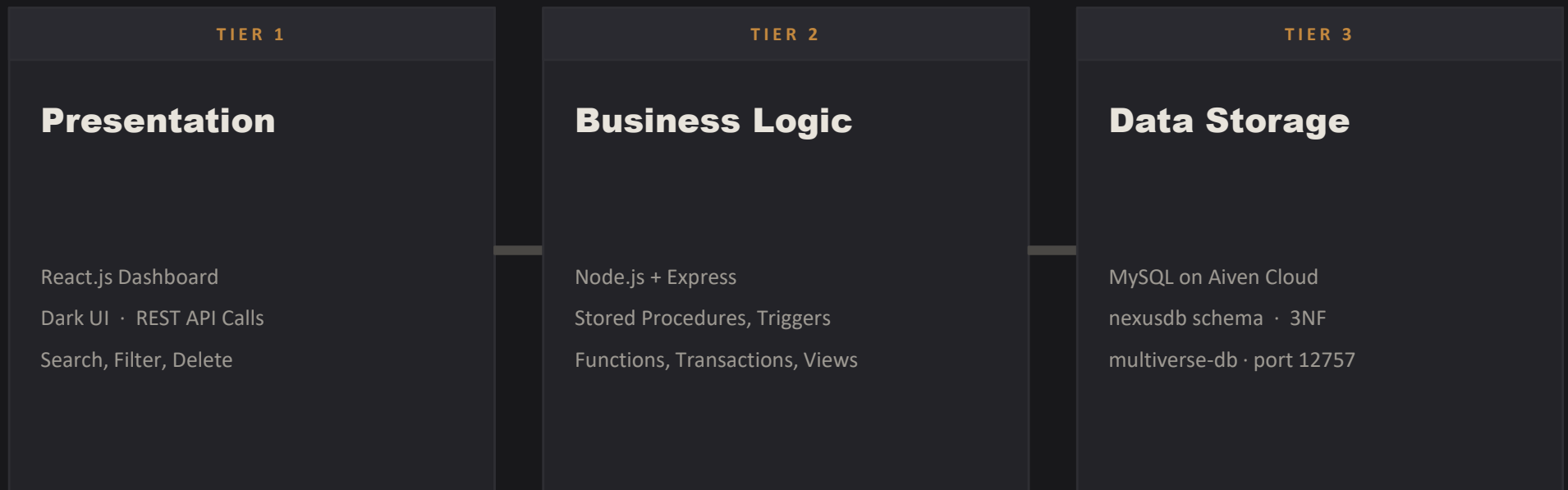
N/A

ifies Indie /

252 listings auto-generated across Steam, Epic, PS5, Xbox, and Switch.

4-page React dashboard: Dashboard, Intelligence Hub, Tier Classifier, Atomic Registration.

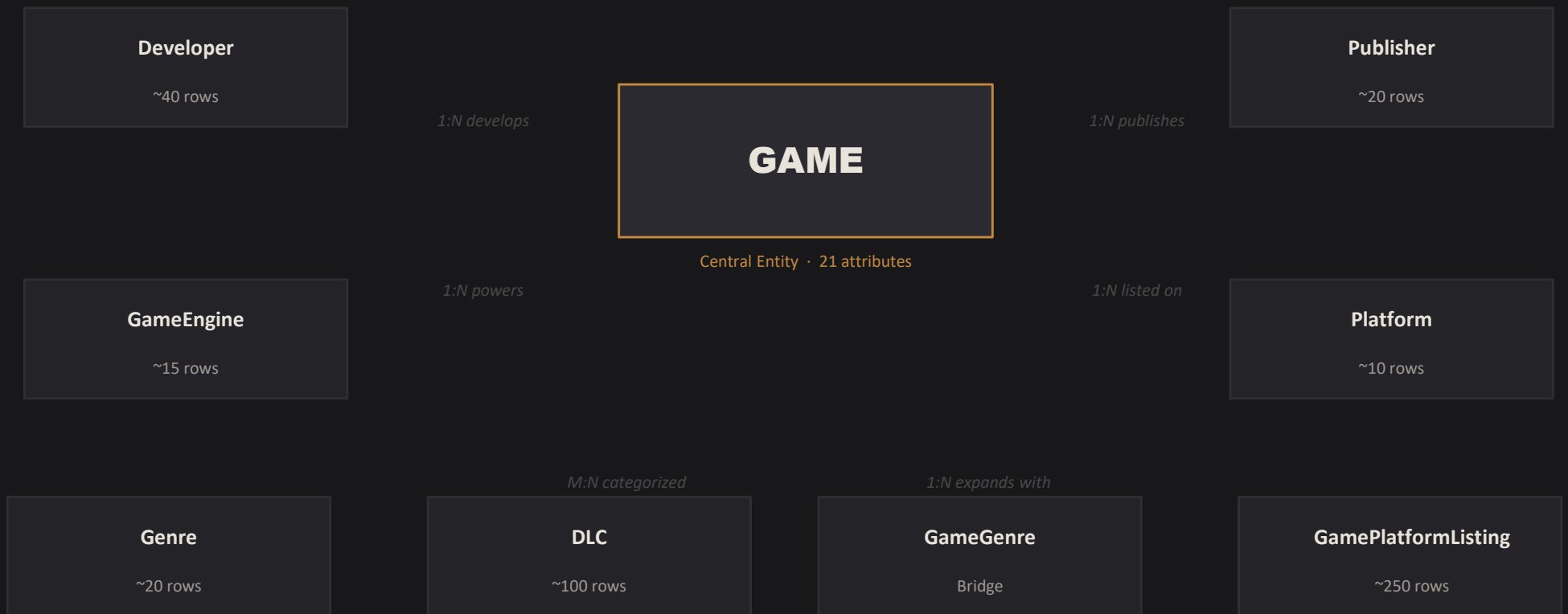
3-Tier Architecture



DATA SOURCES

SteamSpy Top 100 API · Steam Store API (parallel fetch, 10 concurrent) · 252 platform listings auto-generated

Entity Relationship Overview



Schema Decisions

DESIGN DECISIONS

ENUM Types

esrb_rating, publisher_tier — constrained domain values

BOOLEAN Flags

is_free_to_play, is_independent — binary segmentation

Financial Precision

BIGINT for budgets; DECIMAL(5,2) for score fields

CHECK Constraints

metacritic_score validated 0-100 at DB level

ALTER TABLE Demo

cover_image_url added as conceptual DDL walkthrough

INDEXES

idx_game_title

Prevents full table scans on name lookups

idx_release_date

Accelerates time-series range queries

idx_metacritic

Enhances quality-based sorting

NORMALIZATION

1NF — Multi-valued data resolved into bridge tables

2NF — Full functional dependency on composite keys

3NF — Publisher tiers in Publisher table, not Game

Data Manipulation & ACID

INSERT

via RegisterGameComplete

Atomic stored procedure inserts across all related tables in a single transaction.

DELETE

CASCADE on child records

GameGenre → GamePlatformListing → DLC → Game. Referential integrity preserved.

UPDATE

Post-insert enrichment

Stat fields updated after initial registration. ROLLBACK on any failure.

ACID COMPLIANCE

Atomicity

RegisterGameComplete rolls back fully on any failure

Consistency

Relational constraints + ENUM/CHECK domain types

Isolation

Stored procedures execute as protected server units

Durability

InnoDB WAL writes committed data to disk

All write operations are wrapped in explicit transactions. ROLLBACK fires on any failure — data is never partially committed.

9 Analytical Views

GameIntelligenceSummary

Multi-table JOIN · 4 tables

Core game inventory with developer, publisher, engine details

GameROI

Scalar Function Call

CalculateROI() wired directly into this view per row

AboveAverageGames

Subquery · WHERE > AVG(score)

Filters games scoring above the dataset average

TopDevelopers

Correlated Subquery · nested AVG

Isolates studios with above-average revenue records

GenrePerformance

GROUP BY · AVG aggregates

Market trends by primary genre classification

EngineIntelligence

AVG · COUNT

License type vs quality metrics across engines

PublisherImpact

GROUP BY · tier analysis

Tier-based performance aggregation by publisher

DLCMonetization

LEFT JOIN · SUM

Total expansion revenue per base game

PlatformPricing

Multi-table JOIN

Cross-storefront listing details with exclusivity flags

Business Logic Layer

ClassifyGameTier()

Tier Classification

Weighted scoring: budget + team size + publisher tier + revenue → outputs Indie / AA / AAA with component breakdown.

CalculateROI()

Scalar Function

DETERMINISTIC. Formula: $((\text{Revenue} - \text{Budget}) / \text{Budget}) \times 100$. Centralizes ROI across all views.

GetGenreOpportunity()

Market Gap Analysis

Aggregates revenue and copy sales within budget/team constraints. Identifies underserved profitable genres.

GetDeveloperBenchmark()

Peer Comparison

HAVING clause — ranks peers within $\pm 50\%$ budget range by Metacritic score for studio benchmarking.

GetDLCStrategy()

Monetization Patterns

Analyzes top-rated DLC by genre and price to advise on expansion strategies.

RegisterGameComplete()

Atomic Transaction

Full ACID-compliant registration: INSERT → VALIDATE → ASSIGN → COMMIT or full ROLLBACK.

4 Automated Guards

BEFORE INSERT

before_game_insert

F2P Auto-Flag

If base price = 0, automatically sets is_free_to_play. Eliminates manual tagging errors.

BEFORE INSERT / UPDATE

validate_metacritic

Score Validation

Raises SIGNAL SQLSTATE 45000 if score falls outside 0-100. Descriptive error for developers.

BEFORE INSERT

prevent_negative_price

Price Guard

Rejects any negative price entry in GamePlatformListing. Guards revenue analytics integrity.

AFTER INSERT

auto_subscription_flag

Subscription Sync

If a listing includes a subscription, the parent Game record is auto-updated for consistency.

How the Data Was Built

01

SteamSpy API

Top 100 Forever endpoint
Fetches owners, price, tags,
developer

02

Steam Store API

Parallel fetch — 10 concurrent
Release dates, descriptions

03

Revenue Model

F2P tiered by owner count
Paid: 80% effective price
Pre-2010 legacy: $30\% \times \$8$

04

Budget Tier

Estimated by copies sold
Indie / AA / AAA thresholds
automatically assigned

05

Platform Listings

252 cross-platform entries
auto-generated for Steam,
Epic,
PS5, Xbox, Switch

4-Page React Dashboard

01

Dashboard

Live stats & counters · Cover art grid · Search + filter · Delete with cascade

02

Intelligence Hub

ROI table from GameROI view · Above-average performers ·
Engine/Genre/Publisher breakdown · Platform pricing comparison

03

Tier Classifier

Calls `ClassifyGameTier()` · Shows weighted score breakdown · AAA / AA / Indie
output · Auditable component scores

04

Atomic Registration

Calls `RegisterGameComplete()` · Live transaction demo · COMMIT or ROLLBACK
visible · Real-time confirmation

Queries in Action

ROI FUNCTION CALL

```
SELECT title,  
       CalculateROI(revenue_est_usd, dev_budget_usd)  
       AS roi_pct  
FROM Game  
ORDER BY roi_pct DESC LIMIT 10;
```

ABOVE-AVERAGE SUBQUERY

```
SELECT title, metacritic_score  
FROM Game  
WHERE metacritic_score >  
      (SELECT AVG(metacritic_score)  
       FROM Game);
```

SAMPLE RESULTS

Dota 2

~\$8.2B est. revenue

Black Myth: Wukong

ROI > 1,200%

RPG Genre Avg

Metacritic 84.5 / 100

Indie (Unreal Engine)

Higher ROI % than AAA

F2P threshold

0 owners → auto is_free_to_play

What We Delivered

- 100 real Steam games, 9 normalized tables
- 9 views, 5 stored procedures, 1 scalar function
- 4 triggers, 3 B-Tree indexes
- Fully functional live web dashboard
- Real-world revenue modelling with academic SQL coverage
- ACID-compliant atomic transactions verified end-to-end

FUTURE SCOPE

- Metacritic API integration
- ML-based sales prediction
- Live-ops telemetry (DAU / MAU)
- Social sentiment analysis
- User authentication system
- Export to CSV / PDF
- Global cloud scaling (AWS RDS)